

Current Movie Data to Support HappyMovies

Robert Kindle, Ethan Vukovic, Chris Lewis, Ty Lundgren

Course: ISA 345

Instructor: Arthur Carvalho

Date: 5/8/2026

Project Overview.....	3
Database Design & Implementation.....	4
Deployment.....	16
Conclusions & Lessons Learned.....	17

Project Overview

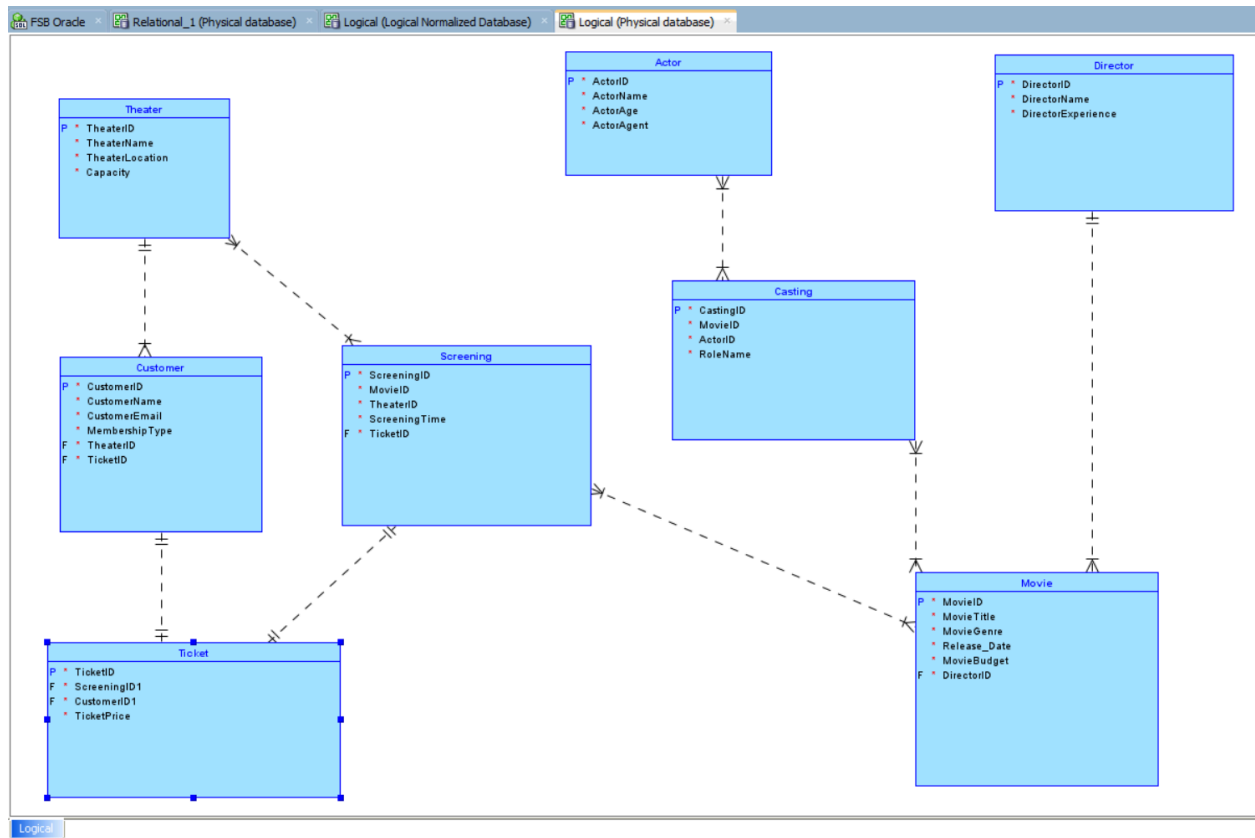
For this project we are designing a database for a movie theater system. The purpose of this system is to manage movies, theaters, customers, and all the main operations that go on in a theater business. This includes things like scheduling movies, selling tickets, and keeping track of actors in films. The database helps keep everything organized and makes sure all the information is stored correctly and easy to access.

Ticket purchases (customer buys ticket for screening): This happens when a customer selects a specific movie showing and purchases a ticket. The system records which customer is buying the ticket, which is important for tracking sales and linking customers to specific shows.

Movie screening schedule (company schedules a movie at a theater): This process is used when the company sets up a new movie showing. It involves assignment of a movie to a specific theater and time. This helps the business stay organized and avoid scheduling conflicts.

Actor assignment (actor is cast in a movie): This occurs when an actor is assigned to a movie along with their role. The system stores which actor is in which movie and what character they are playing. This keeps casting information accurate and easy to manage. Overall, these transactions help the database run the main parts of the business. Each one represents a real action in the system and keeps all the data organized and connected.

Database Design & Implementation



After removing duplicate values, this is the first logical database.

Entity and Attribute Explanation -

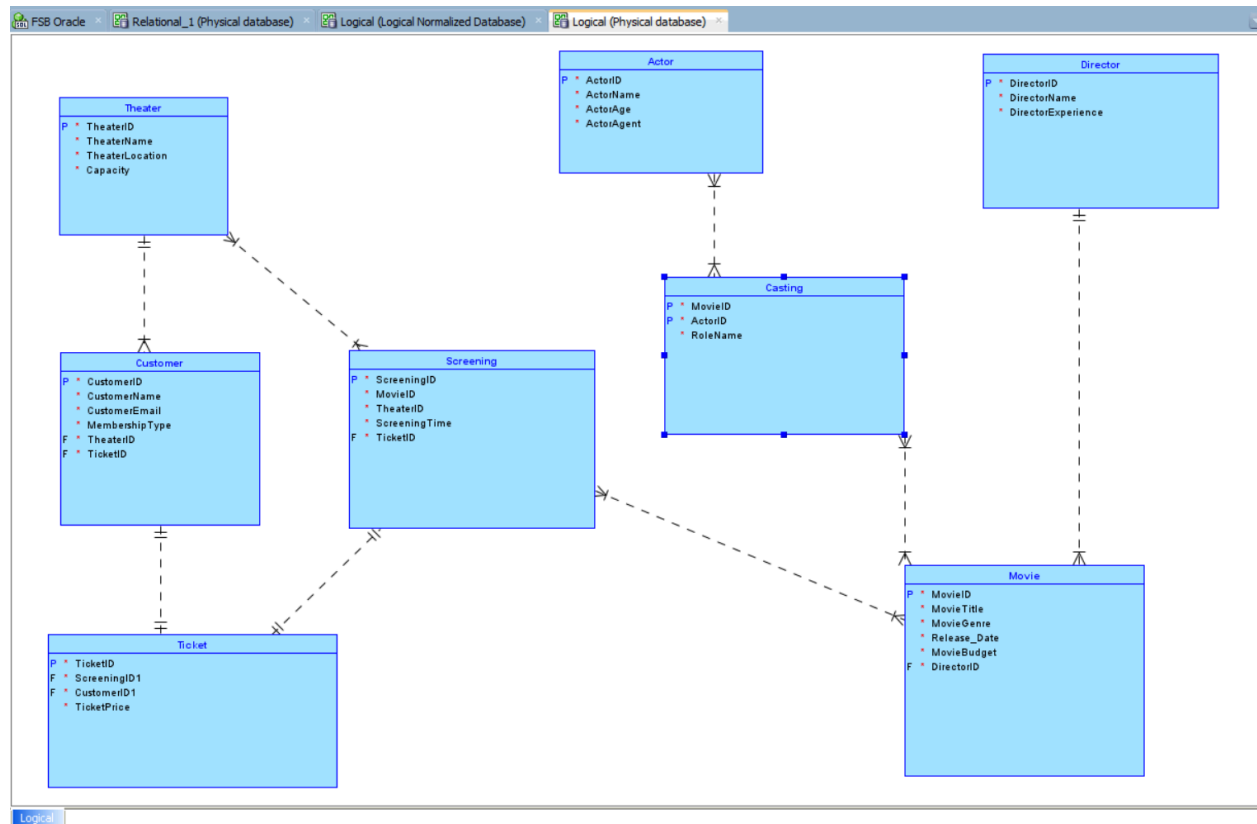
- **Movie** - Shows us the information about each movie, including the directors, casted actors, and the theater/screening of the movie.
 - **MovieID** - Primary key, identifies the movie
 - **MovieTitle** - The Movie Title
 - **MovieGenre** - The genre of the movie (Comedy, Romance, Thriller)
 - **Release_Date** - The Movie Release Date
 - **MovieBudget** - The budget of the movie
- **Casting** - The bridge for Movie and Actors, shows us which actor is in which movie.
 - **CastingID** - Unique casting Number
 - **MovieID** - Unique ID that shows us which movie is being cast
 - **ActorID** - Unique ID that shows us which actor was cast
 - **RoleName** - The role that actors are given when they are cast to a movie
- **Ticket** - Shows us the information about which customer is buying a certain ticket for each screening.
 - **TicketID** - Unique ID given to a ticket when purchased
 - **ScreeningID** - Unique ID that shows us which ticket belongs to what screening
 - **CustomerID** - Unique ID that shows us which customer buys what ticket
 - **TicketPrice** - Price of the ticket that is purchased
- **Screening** - The bridge between theater and movie, showing us the time and screening of specific movies at certain theaters.

- **ScreeningID** - Unique ID given to each screening of a movie
- **MovieID** - Unique ID that shows us what movie is being screened
- **TheaterID** - Unique ID that shows us which theater is screening a movie
- **ScreeningTime** - What time the movie is being screened
- **Customer** - Shows us information about customers, including basic information and their membership at their theater.
 - **CustomerID** - Unique ID for each customer
 - **CustomerName** - The name of the Customer
 - **CustomerEmail** - Email for the Customer
 - **MembershipType** - The type of Membership that the customer has (Gold, Silver, Bronze)
- **Theater** - Shows us information about theaters, including any movies they may be screening.
 - **TheaterId** - Unique Identification for each Theater
 - **TheaterName** - Name of the Theater
 - **TheaterLocation** - Location of the Theater
 - **Capacity** - The max number of people the theater can hold
- **Actor** - Shows us information about potential actors to be casted into movies.
 - **ActorID** - Unique Identifier for a certain Actor
 - **ActorName** - Name of the Actor
 - **ActorAge** - Age of the Actor
 - **ActorAgent** - Agent that the Actor has hired

- **Director** - Shows us information about about the Directors and what movies they are apart of
 - **DirectorID** - Unique Identifier for a certain Director
 - **DirectorName** - Name of Director
 - **DirectorExperience** - Years of Experience the Director has

Relationships -

- The main table in the model is Movie. One director can make many movies, one movie belongs to one director. This has to be mandatory because a Movie has to have a director in order to be filmed.
- Actors have to be cast to be in the movie. Many actors can be casted to many movies, with movies having many actors. This relationship is mandatory because in order for the movie to happen there must be Actors in the movie.
- A movie must be screened at a theater. Many movies can be screened at many theaters, with theaters screening many movies. Tickets must be purchased for each screening. A customer can buy one ticket, and one ticket can be used for one screening. This has to be mandatory because there has to be movies, so we can screen the movies at the theater.
- Customers must visit a theater to watch a screening. One customer can visit one theater at a time. Theaters can have many customers. This is mandatory because in order to be in business, customers need to go to the theater.



Our normalized database. To remove redundancies, we eliminated the casting ID since movie ID and actor ID already relate to casting. All other tables were basically normalized so the normalization process was simple.

Functional Dependences:

MovieID → MovieTitle, MovieGenre, Release_Date, MovieBudget, DirectorID

A film's unique MovieID dictates its title, genre and when it hits theaters, the cost and exactly what director directed it. Works because the MovieID is the Primary key. There's no overlap. For this we had to remove DirectorName and Director Experience from this table.

CustomerID → CustomerName, CustomerEmail, MembershipType

- A customer's unique ID will determine the name, email, and membership type associated with the customer.

DirectorID → DirectorName, DirectorExperience

- A directorsID has one specific filmmaker and their years of experience.

ActorID → ActorName, ActorAge, ActorAgent

- An actors ID explains the Name, Age, and Agent associated with the Actor

TheaterID → TheaterName, TheaterLocation, Capacity

- Each TheaterID is associated with a TheaterName, location and a specific capacity for the theater

(TicketID, ScreeningID, CustomerID) → TicketPrice

(ScreeningID, MovieID, TheaterID) → ScreeningTime

(MovieID, ActorID, CastingID) → RoleName

Operational SQL

1. Contain a subquery

```
SELECT MovieID, MovieTitle, MovieGenre, MovieBudget  
FROM movie  
WHERE MovieBudget > (SELECT AVG(MovieBudget) FROM movie)  
ORDER BY MovieBudget DESC;
```

This query adds real value by bringing a risk and financial analysis perspective into the database. Instead of treating every film the same it uses a subquery to find the average production budget and highlights only the films that exceed it. It reveals the higher budget films, which can be more of a financial risk for businesses and for them to succeed need to have a stronger performance than average. Storing them from highest to lowest just gives a better idea of what's happening.

2. Contain an inner join

```
SELECT MovieTitle, SUM(TicketPrice) AS "Total Revenue"  
FROM MOVIE INNER JOIN SCREENING ON MOVIE.MovieID = SCREENING.MovieID  
INNER JOIN TICKET ON SCREENING.ScreeningID = TICKET.ScreeningID  
GROUP BY MovieTitle;
```

Displays total revenue that a Movie earns based on the total price of all tickets sold which will help a business determine which movies are performing the best. Using this query a business could properly allocate future funding based on past revenues or provide incentives for well performing movies.

3. Contain an outer join

```
SELECT SCREENING.ScreeningID, MOVIE.MovieTitle, COUNT(TICKET.TicketID) AS TicketsSold  
FROM SCREENING JOIN MOVIE ON SCREENING.MovieID = MOVIE.MovieID  
LEFT JOIN TICKET ON SCREENING.ScreeningID = TICKET.ScreeningID  
GROUP BY screening.screeningid, MOVIE.MovieTitle;
```

This query will display the ScreeningID, MovieTitle and TicketsSold. This will allow you to see just how many tickets are being bought per Movie/Screening. Not only that but it will also show you which of the screenings are poorly performing(or unused) allowing you to adjust pricing and promotions or optimize the screening schedule overall.

4. Contain a GROUP BY clause without the HAVING clause

```
SELECT TheaterName, TheaterLocation, SUM(TicketPrice) AS "Revenue for Location"  
FROM THEATER JOIN SCREENING ON THEATER.TheaterID = SCREENING.TheaterID  
JOIN TICKET ON SCREENING.ScreeningID = TICKET.ScreeningID  
GROUP BY TheaterName, TheaterLocation;
```

This query will return the Theatre Name, Location, and revenue earned at that location. This will help businesses determine which theaters are performing the best and which locations/cities would be optimal for new theaters. This can be helpful for managers when deciding where to expand and which location may need improvement or more marketing.

5. Group by with having clause

```
SELECT customer.customername, SUM(ticketprice)
FROM customer, ticket
WHERE customer.customerid = ticket.customerid
GROUP BY customername
HAVING SUM(ticketprice) > 15.00;
```

This query identifies your higher-value customers by calculating the total amount each person has spent on tickets and filtering for those who go over the threshold of \$15.00. This could be used for target marketing and loyalty programs. By figuring out the customers who spend above a certain amount a theater manager can offer them exclusive VIP rewards because these are the higher spenders. IT allows them to focus it's limited marketing resources on the preferred target market.

Indexes:

```
CREATE INDEX ticket_price_index ON TICKET(TicketPrice);
```

Ticket price index improves the performance when filtering or sorting by price. Also because it is common and useful for revenue analysis and running pricing relating reports. Just makes it a lot more efficient.

```
CREATE INDEX moviebudget_index ON MOVIE(MovieBudget);
```

This index supports financial analysis by speeding up queries that compare movie budgets or identify high-budget films. This is important for assessing risk and understanding where the most money is being invested.

CREATE INDEX screeningtime_index ON SCREENING(screeningtime);

This index makes it faster to search and organize movie showtimes. It is useful for scheduling, avoiding conflicts, and quickly retrieving available screenings for users of operations.

CREATE INDEX capacity_index ON THEATER(capacity);

This index helps with the queries related to theater size and attendance. IT supports better decision making when assigning movies to theaters and ensures theaters are used more efficiently based on capacity.

View:

CREATE VIEW nonVIP AS

**SELECT MovieID, MovieTitle, MovieGenre, Release_Date, MovieBudget
FROM movie;**

This function creates a view for non VIP customers for the table Movie, restricting their access to movie data. When a customer has a membership type of VIP, they have complete access to the movie data.

Trigger:

```
CREATE OR REPLACE TRIGGER prevent_overbooking
BEFORE INSERT ON ticket
FOR EACH ROW
```

```
DECLARE
    ticket_count NUMBER;
    theater_capacity NUMBER;
```

```
BEGIN
```

```
    SELECT COUNT(*)
    INTO ticket_count
    FROM ticket
    WHERE screeningid = :NEW.screeningid
        AND theaterid = :NEW.theaterid
        AND movieid = :NEW.movieid;
```

```
    SELECT capacity
    INTO theater_capacity
    FROM theater
    WHERE theaterid = :NEW.theaterid;
```

```
    IF ticket_count >= theater_capacity THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'The theater is full. Cannot book more tickets.'
        );
    END IF;
```

```
END;
/
```

This trigger helps us prevent overbooking at the theater. The process begins when the customer buys the ticket. The trigger goes and looks up the number of tickets that are already sold and the maximum number of people that can be in the theater. If tickets are already sold out or the maximum amount of people have already been reached, and RAISE_APLLLICATION_ERROR will show for the user trying to order the ticket. This ensures the theater from overbooking their movies.

Deployment

The link below features our SQL Agent that connects to our Movie database and can run user-inputted queries:

<https://miamiuniversity.app.n8n.cloud/webhook/dd8cb353-9770-40a7-970b-561ef511bd81/chat>

This web application allows directors to easily manage their actors by filtering them through specific measures, as well as allowing directors to add and delete actors to their data (changes the data in the Oracle database):

<https://ai.studio/apps/210b6018-0e99-410e-82a5-544bc7d872ca>

Conclusions & Lessons Learned

The “Happy Movies” database system gives the company a more organized and efficient way to manage theater operations and important business data by keeping movie schedules, customer transactions, employee information, and talent rosters all in one place. Managers from Happy Movies can equally access reliable information whenever it is needed. Our SQL queries and reports help identify which theater locations are performing the best, track ticket revenue, and recognize VIP customers. This allows the company to make better decisions about marketing, customer engagement, and possible future investments.

Throughout this project we used BCNF to make sure our database had no redundancies in it to make sure every attribute depended on only one primary key. This was definitely tricky because we had so many tables, so we had to take our time with this and do it right. A few errors occurred in our database during development, mainly with the database modeling portion. During the normalization process, a few attributes were being duplicated by the system, and unnecessary composite primary keys were placed. After fixing the logical database and engineering it into a physical database, the original DDL code was faulty so we had to manually refurbish the physical database model. After that, inserting the data was incorrect, so we manually changed the formatting for that as well.

All in all, we learned how important it is to balance strong technical features with ease of use, from concepts and applications we used in class for the design. Throughout the project we applied what we learned about database design, normalization, queries, and data integrity to improve the system. In the end, we created an efficient, and scalable database system that successfully met the goals of the project.